

STUDYMATE — AI CHATBOT
PROJECT DOCUMENTATION
INTERNSHIP TASK 04 – AI / ML

Name: Shazmeen

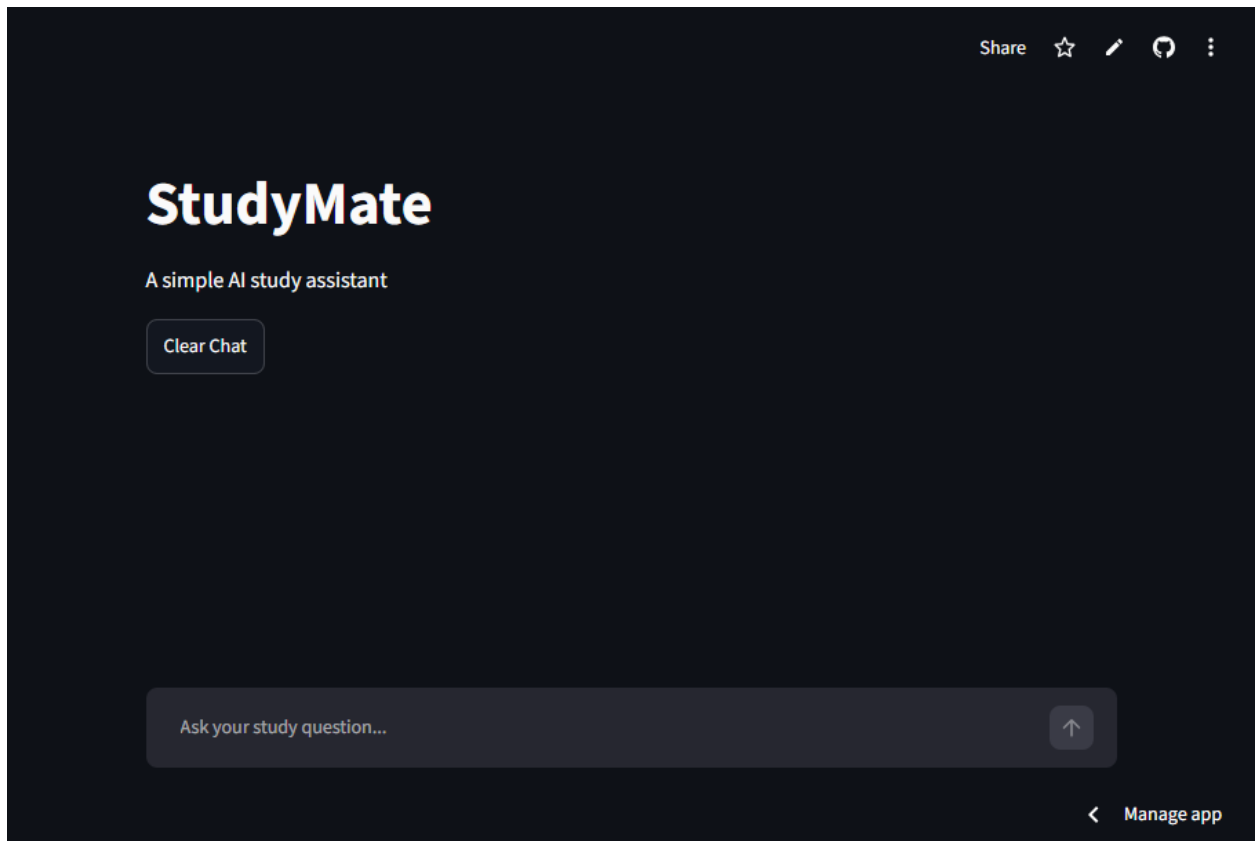
Program: AI / ML Internship

1. Project Overview:

StudyMate is a simple AI-powered study assistant developed. The application is designed to help students ask educational questions and receive clear, conversational responses. The chatbot uses the **Gemini API** for response generation and **Streamlit** to provide an interactive web-based interface. The project focuses on practical implementation of conversational AI concepts, including API integration, prompt engineering, query processing, response generation, conversation management, frontend development, deployment, and testing.

Project Highlights:

- Educational chatbot for study-related questions
- Gemini API integration
- Interactive Streamlit interface
- Conversation history within a chat session
- Clear Chat functionality
- Public deployment
- Response and usability testing
- GitHub-based source code management



StudyMate AI chatbot interface

2. Project Objective :

The main objective of this project is to design and develop a functional AI-powered chatbot that can understand user questions and generate meaningful responses through an integrated Large Language Model.

The project was developed to gain practical experience in:

- Conversational AI
- Large Language Model integration
- API integration
- Prompt engineering
- Query processing
- Response generation
- Conversation history management
- Interactive UI development
- Application deployment
- System testing and evaluation

3. Chatbot Purpose and Use Case:

1. Purpose:

StudyMate was developed as an **educational study assistant**. The purpose of the chatbot is to provide students with a simple interface where they can ask questions related to their studies and receive understandable responses. Instead of requiring a student to navigate through multiple resources for a basic explanation, StudyMate provides a conversational way to ask questions and continue the discussion.

2. Target Users:

The primary target users are:

- University students
- College students
- Learners studying technical subjects
- Beginners who need simple explanations

3. Example Use Cases:

A student can use StudyMate to ask questions such as:

- What is Machine Learning?
- Explain Python functions.
- What is Artificial Intelligence?
- Give me a simple example of a concept.
- Ask a follow-up question about the previous explanation.

The chatbot is intended as a **study support tool**, rather than a replacement for teachers, textbooks, or official academic resources.

4. Technologies Used:

Technology	Purpose
Python	Main programming language
Gemini API	AI model integration and response generation
Google GenAI SDK	Communication with the Gemini API
Streamlit	Interactive web interface
python-dotenv	Environment variable/API key management
GitHub	Source code version control and repository hosting
Streamlit Community Cloud	Public deployment

Why these technologies were selected:

Python was used because it provides a simple environment for implementing API-based AI applications.

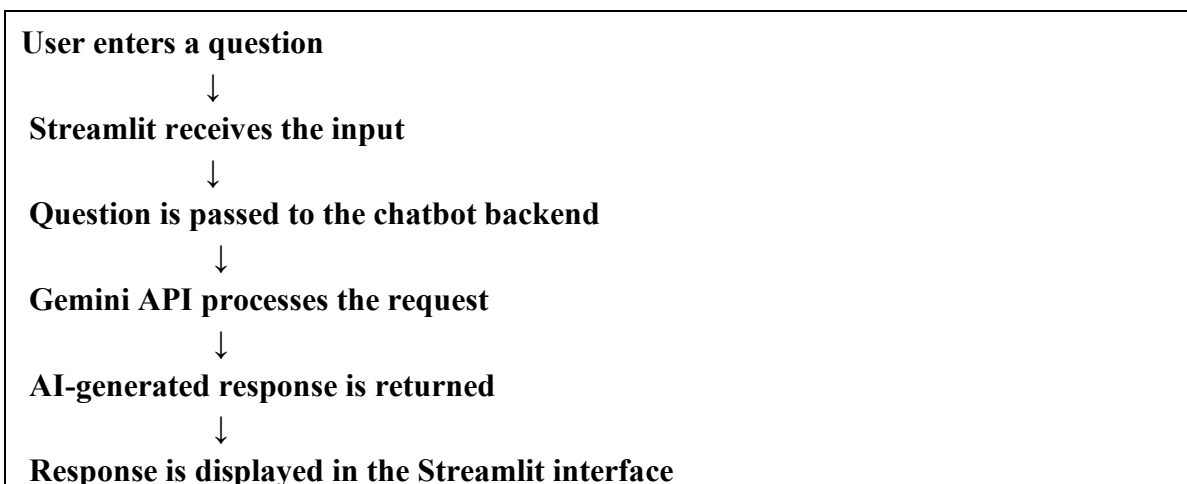
The Gemini API was selected as the AI model provider for generating chatbot responses.

Streamlit was selected for the frontend because it allows a functional interactive web interface to be developed directly with Python.

GitHub was used to maintain and publicly share the source code, while Streamlit Community Cloud was used to make the chatbot accessible through a public link.

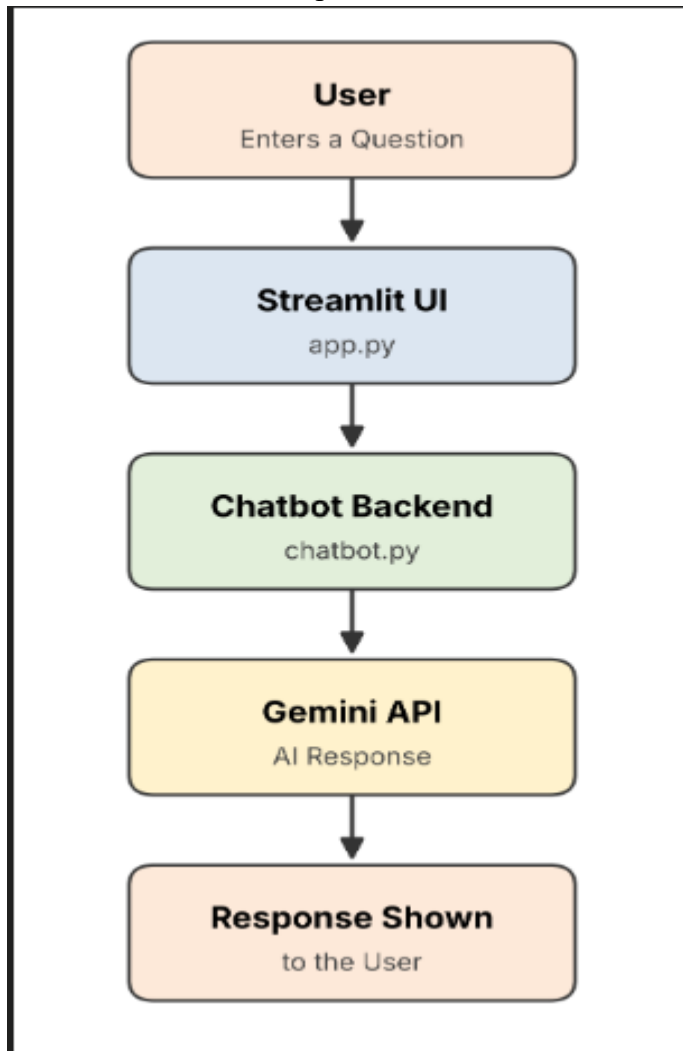
5. System Workflow:

The overall workflow of StudyMate is:



↓
Conversation is stored in the current session

The basic architecture can be represented as:



6. AI Model Integration:

StudyMate uses the **Gemini API** as its AI model. The backend initializes the Gemini client using an API key stored securely as an environment variable rather than directly inside the source code.

The chatbot sends the user's query to the Gemini model and receives the generated response. This implementation demonstrates the integration of an external Large Language Model into a practical chatbot application.

AI Integration Process:

1. Gemini client is initialized.
2. The API key is loaded securely.
3. A chat session is created.
4. The user submits a question.
5. The question is sent to Gemini.
6. Gemini generates a response.
7. The response is returned to the Streamlit application.
8. The response is displayed to the user.

```
1  import os
2  from dotenv import load_dotenv
3  from google import genai
4  from google.genai import types
5
6  load_dotenv()
7
8  api_key = os.getenv("GEMINI_API_KEY")
9
10 if not api_key:
11     raise ValueError("GEMINI_API_KEY was not found in the .env file.")
12
13 client = genai.Client(api_key=api_key)
14
```

Gemini API integration in the chatbot backend

7. Backend Development:

The backend contains the main chatbot logic. It handles the communication between the Streamlit frontend and Gemini API.

The backend implementation:

- Query Processing
- Prompt Engineering
- Response Generation
- Conversation History Management

- **Query Processing:**

When a user enters a question, the Streamlit application captures the input and passes it to the chatbot backend. The backend receives the question and sends it to the active Gemini chat session for processing. This allows the chatbot to convert the user's input into a request that can be handled by the AI model.

```
36 # User input
37 question = st.chat_input("Ask your study question...")
38
39
40 if question:
41
42     # Show user question
43     st.session_state.messages.append({
44         "role": "user",
45         "content": question
46     })
47
48     with st.chat_message("user"):
49         st.write(question)
50
51
52     # Generate response
53     with st.chat_message("assistant"):
54
55         try:
56             answer = get_response(
57                 st.session_state.chat,
58                 question
59             )
60
61             if not answer:
62                 answer = "I could not generate an answer."
63
64             st.write(answer)
65
```

User query processing and AI response generation

- **Prompt Engineering:**

Prompt engineering is used to guide the chatbot's behaviour and response style. A system-level instruction is used to define StudyMate's role as a study assistant and to encourage useful

educational responses. This helps maintain a consistent purpose for the chatbot instead of treating every request as a completely unrestricted interaction.

```
16  system_prompt = ""
17  You are StudyMate, a helpful study assistant.
18
19  Your job is to help students understand study topics
20  in simple and clear language.
21
22  Follow these rules:
23
24  - Explain difficult topics in simple words.
25  - Give examples when they help.
26  - Keep answers organized and easy to read.
27  - Use the previous conversation when answering follow-up questions.
28  - Do not make answers unnecessarily complicated.
29  - If you are not sure about something, say that clearly.
30  ""
31
```

System prompt defining StudyMate's behavior and response guidelines

- **Response Generation:**

After receiving the user's question, the backend sends the request to the Gemini model. The generated response is then returned to the Streamlit application and displayed in the assistant message area. The application also includes error handling so that if an API or runtime error occurs, the user receives an appropriate error message instead of the application failing silently.

- **Conversation History Management:**

```

15     # Start a new chat
16     if "messages" not in st.session_state:
17         st.session_state.messages = []
18
19     if "chat" not in st.session_state:
20         st.session_state.chat = create_chat()
21
22
23     # Clear chat
24     if st.button("Clear Chat"):
25         st.session_state.messages = []
26         st.session_state.chat = create_chat()
27         st.rerun()
28
29
30     # Show previous messages
31     for message in st.session_state.messages:
32         with st.chat_message(message["role"]):
33             st.write(message["content"])
34
35

```

Session state used for conversation history and chat reset

The chatbot manages conversation history using Streamlit session state. User and assistant messages are stored in **st.session_state.messages**, allowing previous messages to remain visible during the conversation. The active chat session is maintained through **st.session_state.chat**. The Clear Chat button resets both the stored messages and the active chat session, allowing the user to start a new conversation.

8. Frontend Development:

The frontend of StudyMate was developed using **Streamlit**. The interface provides a simple and clean environment where users can:

- View the chatbot title
- Enter questions
- View previous messages
- Read AI-generated responses
- Start a new conversation using the Clear Chat button

The interface was intentionally kept simple so that students can focus on asking questions without unnecessary controls.

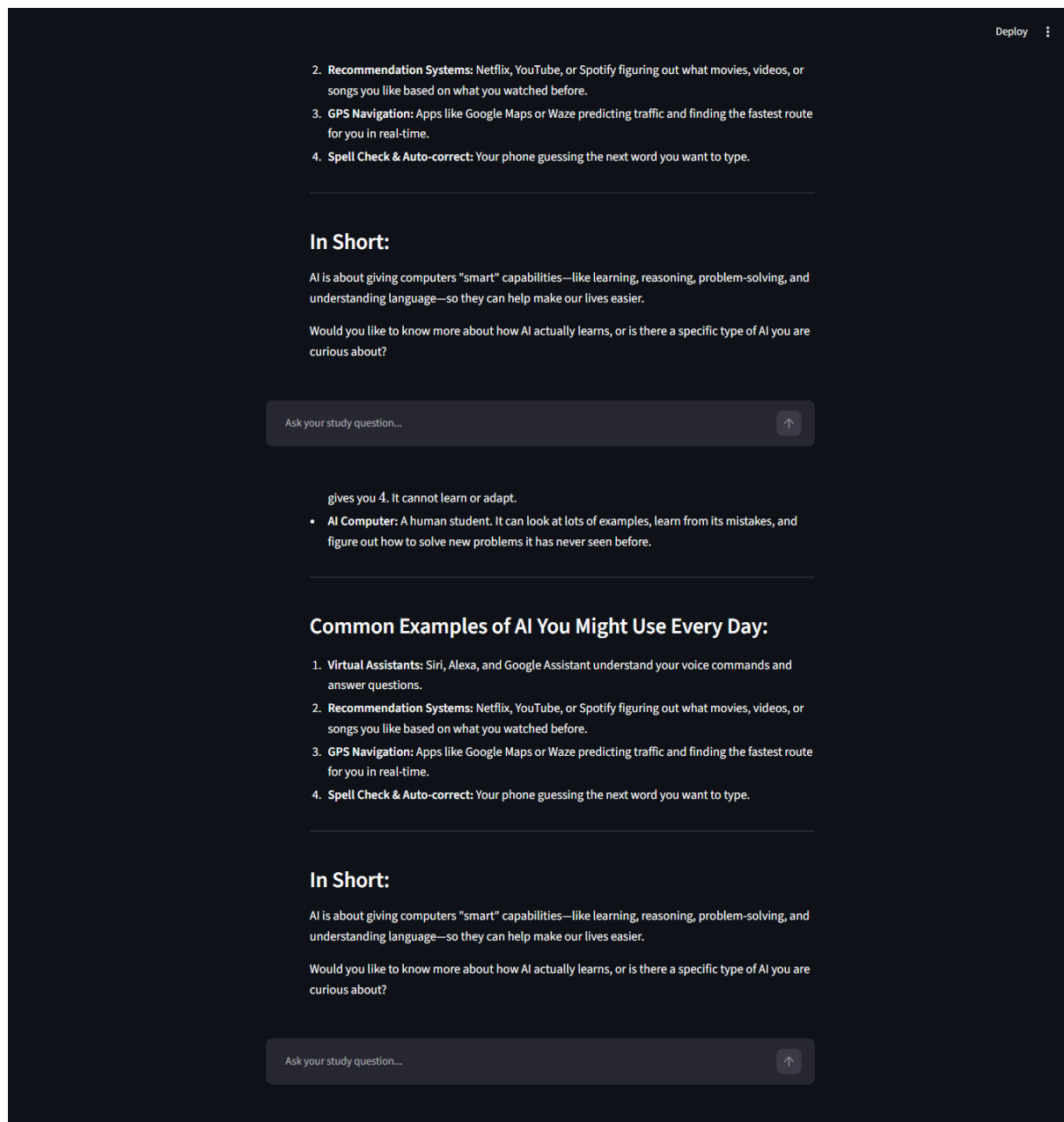
9. Clear Chat Functionality:

A **Clear Chat** button was implemented to allow users to start a fresh conversation.

When the button is clicked:

1. Existing messages are cleared.
2. A new chat session is created.
3. The application is refreshed.

This prevents previous conversation content from remaining when the user wants to start a new topic.



Clear Chat functionality with a new conversation

10. Deployment:

After completing the application locally, StudyMate was deployed online using **Streamlit Community Cloud**. The deployment allows users to access the chatbot through a public web link without running the project locally. The deployed version was opened and tested to verify that the application was accessible and capable of processing user questions.

Deployment Process:

The main deployment steps were:

1. Source code was uploaded to GitHub.
2. A Streamlit Community Cloud application was created.
3. The GitHub repository and application entry point were selected.
4. The Gemini API key was added through Streamlit Secrets.
5. The application was deployed.
6. The public application link was tested.

The application was successfully deployed through Streamlit Community Cloud and made accessible through a public link.

<https://studymatechatbot-psqdmrdfyzahnc29gm7z3c.streamlit.app/>

11. Testing and Evaluation:

- **Response Accuracy:**

The chatbot was tested using normal educational questions to determine whether it produced relevant and understandable answers.

Test Example:

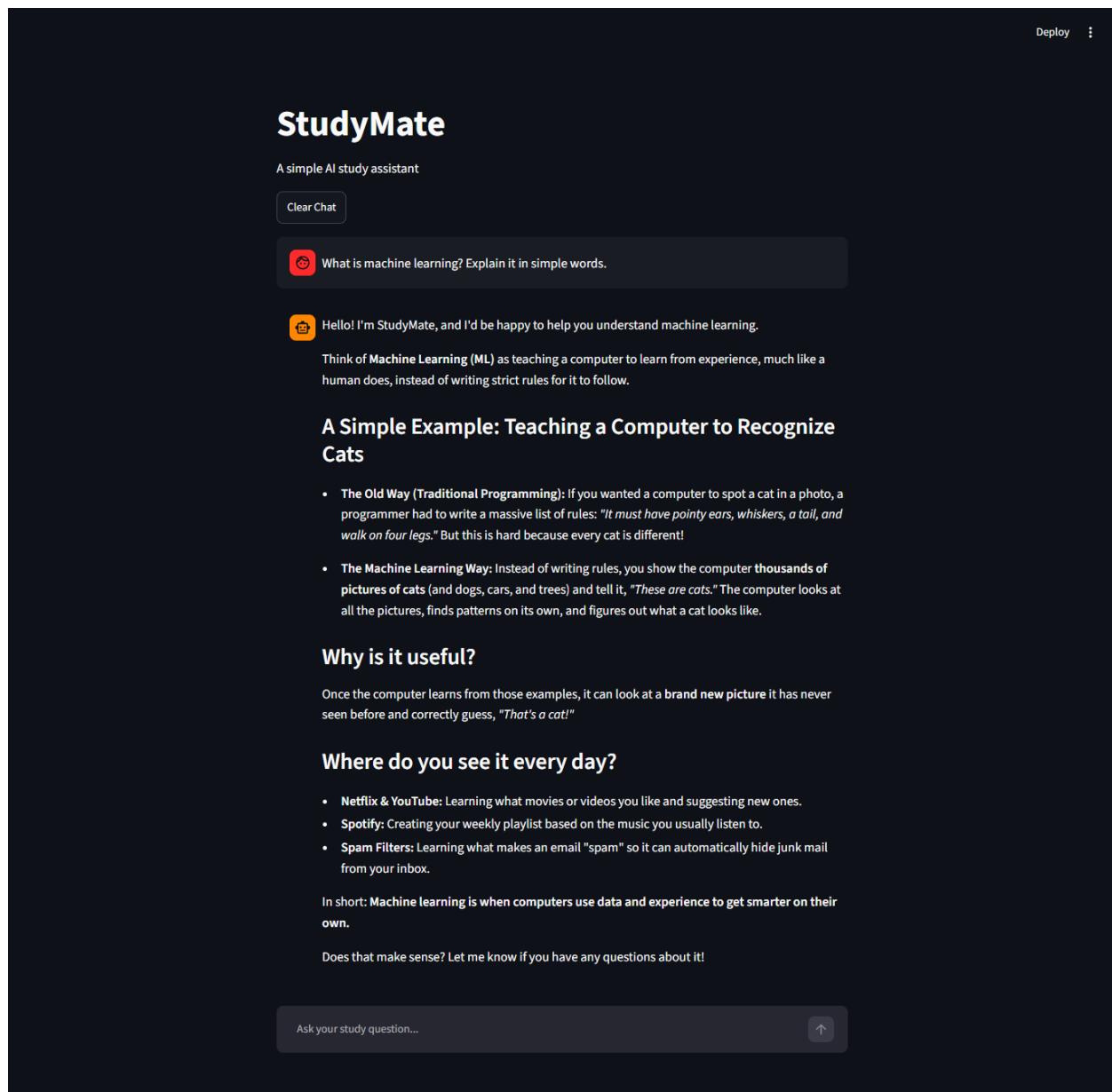
Question:

What is machine learning? Explain it in simple words.

The chatbot generated a relevant explanation suitable for a beginner-level learner.

Result:

The response was considered satisfactory for the intended educational use case because it directly addressed the question and provided an understandable explanation.



Response accuracy test using an educational question

- **User Experience:**

The interface was evaluated from the user's perspective.

The main aspects considered were:

- Ease of entering a question
- Readability of responses
- Simplicity of the interface
- Visibility of previous messages

- Accessibility of the Clear Chat feature

The interface provides a straightforward interaction where users can enter a question through the chat input and immediately view the response.

Result:

The overall interface was found to be simple and easy to use for the intended student audience.

- **Conversation Quality:**

Conversation quality was evaluated by asking a follow-up question within the same conversation.

Test:

Initial question:

What is machine learning?

Follow-up:

Can you give me a simple example?

The chatbot was able to continue the conversation and respond to the follow-up request based on the ongoing chat context.

Result:

The conversation flow was satisfactory for the intended use case, as the chatbot could handle a follow-up question without requiring the user to repeat the complete context.

StudyMate

A simple AI study assistant

Clear Chat

 What is machine learning? Explain it in simple words.

 Hello! I'm StudyMate, and I'd be happy to help you understand machine learning.

Think of **Machine Learning (ML)** as teaching a computer to learn from experience, much like a human does, instead of writing strict rules for it to follow.

A Simple Example: Teaching a Computer to Recognize Cats

- **The Old Way (Traditional Programming):** If you wanted a computer to spot a cat in a photo, a programmer had to write a massive list of rules: *"It must have pointy ears, whiskers, a tail, and walk on four legs."* But this is hard because every cat is different!
- **The Machine Learning Way:** Instead of writing rules, you show the computer **thousands of pictures of cats** (and dogs, cars, and trees) and tell it, *"These are cats."* The computer looks at all the pictures, finds patterns on its own, and figures out what a cat looks like.

Why is it useful?

Once the computer learns from those examples, it can look at a **brand new picture** it has never seen before and correctly guess, *"That's a cat!"*

Where do you see it every day?

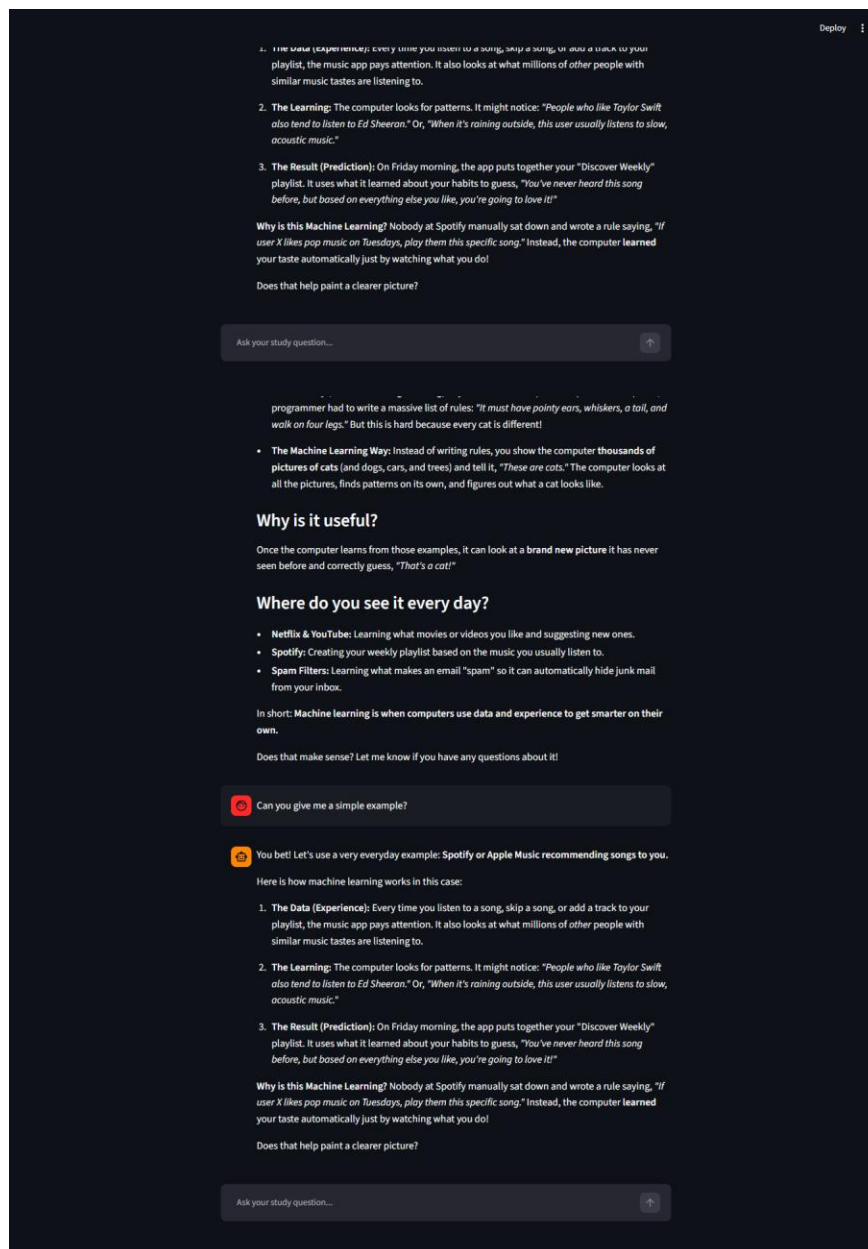
- **Netflix & YouTube:** Learning what movies or videos you like and suggesting new ones.
- **Spotify:** Creating your weekly playlist based on the music you usually listen to.
- **Spam Filters:** Learning what makes an email "spam" so it can automatically hide junk mail from your inbox.

In short: **Machine learning is when computers use data and experience to get smarter on their own.**

Does that make sense? Let me know if you have any questions about it!

Ask your study question...





Follow-up question demonstrating conversational continuity

- **System performance:**

System performance was evaluated by observing the response time during testing.

The recorded response times were approximately:

Test	Response Time
Test 1	6 seconds
Test 2	4 seconds
Average	5 seconds

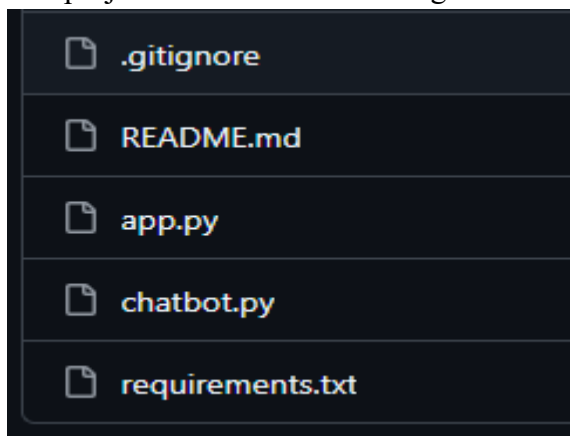
The average response time was approximately **5 seconds** across the recorded tests.

Result:

The response time was considered acceptable for this project because the application successfully processed the requests and returned responses within a few seconds.

12. Project Structure:

The project contains the following main files:



StudyMate GitHub repository structure

File Description:

File	Purpose
app.py	Streamlit frontend and application flow
chatbot.py	Gemini API integration and chatbot logic
requirements.txt	Required Python packages
.gitignore	Prevents sensitive/unnecessary files from being uploaded
README.md	Project documentation and setup instructions

13. Security and API Key Management:

The Gemini API key is treated as sensitive information and is not included directly in the GitHub source code.

The project uses environment-based configuration for local development, while the deployed application uses **Streamlit Secrets**.

The **.gitignore** file includes:

```
1  venv/  
2  .env  
3  __pycache__/
```

This prevents the local **.env** file and unnecessary Python files from being committed to the public repository.

The API key was also kept hidden in the deployed application's secret configuration.

14. Results:

The completed StudyMate application successfully achieved the main objectives of the task.

The final system provides:

- A functional AI-powered chatbot
- Gemini API integration
- Educational question answering

- Prompt-based chatbot behaviour
- Conversation history
- Follow-up conversation support
- Clear Chat functionality
- Interactive Streamlit interface
- Public deployment
- GitHub source code
- Testing across the four required evaluation areas

The project demonstrates a practical implementation of conversational AI using an external Large Language Model and a lightweight web interface.

15. Limitations:

Although StudyMate is functional, it has some limitations.

1. Dependence on an external API:

The chatbot depends on the availability of the Gemini API. If the API is unavailable or the configured quota is exhausted, the chatbot may not be able to generate responses.

2. General-purpose educational responses:

StudyMate is designed as a general study assistant. It does not provide specialized academic verification for every response.

3. No persistent user accounts:

The current version does not include user authentication or permanent user profiles.

4. Session-based conversation:

Conversation history is maintained within the active application session rather than being stored as a permanent database record.

16. Future Improvements:

Several improvements could be added in future versions:

- User authentication
- Persistent conversation storage
- Subject-specific study modes
- PDF/document-based question answering

- Retrieval-Augmented Generation (RAG)
- Quiz generation
- MCQ generation
- Study-plan generation
- Voice input and output
- Improved response evaluation
- More advanced personalization

These improvements could make StudyMate more useful as a complete student learning assistant.

17. Conclusion:

The project demonstrates the complete workflow of developing a simple conversational AI application, starting from defining a practical use case and integrating an AI model to implementing backend functionality, creating an interactive frontend, deploying the application, and evaluating its performance. The chatbot successfully handles educational questions, maintains conversation context within a session, provides a clear user interface, and is accessible through a public deployment. Through this project, practical experience was gained in **Gemini API integration, prompt engineering, Python-based chatbot development, Streamlit UI development, conversation management, deployment, and system testing**. Overall, StudyMate provides a functional foundation for a student-focused AI study assistant and can be further expanded with features such as RAG, document-based learning, quizzes, personalization, and persistent conversation history.